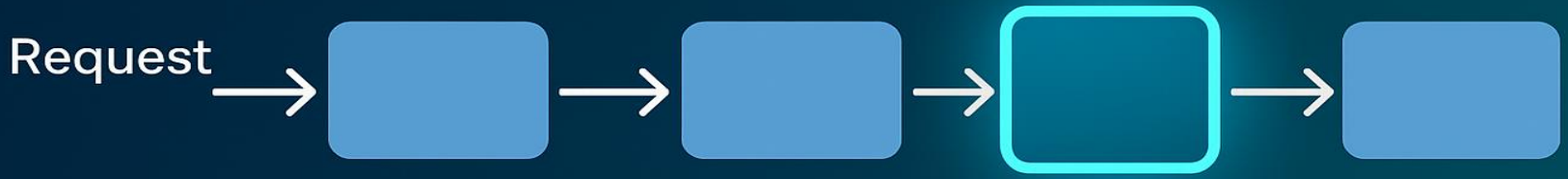
Chain of Responsibility Design Pattern

A Behavioral Design Pattern in OOP



Creational (5)

- [x] Factory
- [ ] Abstract Factory
- [x] Singleton
- [ ] Prototype
- [ ] Builder

Structural (7)

- [x] Adapter
- [ ] Bridge
- [ ] Composite
- [x] Decorator
- [x] Facade
- [ ] Flyweight
- [x] Proxy

Behavioral (11)

-  Chain Of Responsibility
- [ ] Command
- [ ] Interpreter
- [ ] Iterator
- [ ] Mediator
- [ ] Memento
- [ ] Observer
- [ ] State
- [ ] Strategy
- [x] Template
- [ ] Visitor

Chain of Responsibility



Chain of Responsibility Pattern

A Behavioral pattern

Avoid coupling the sender of a request to its receiver by giving more than one object to handle request.

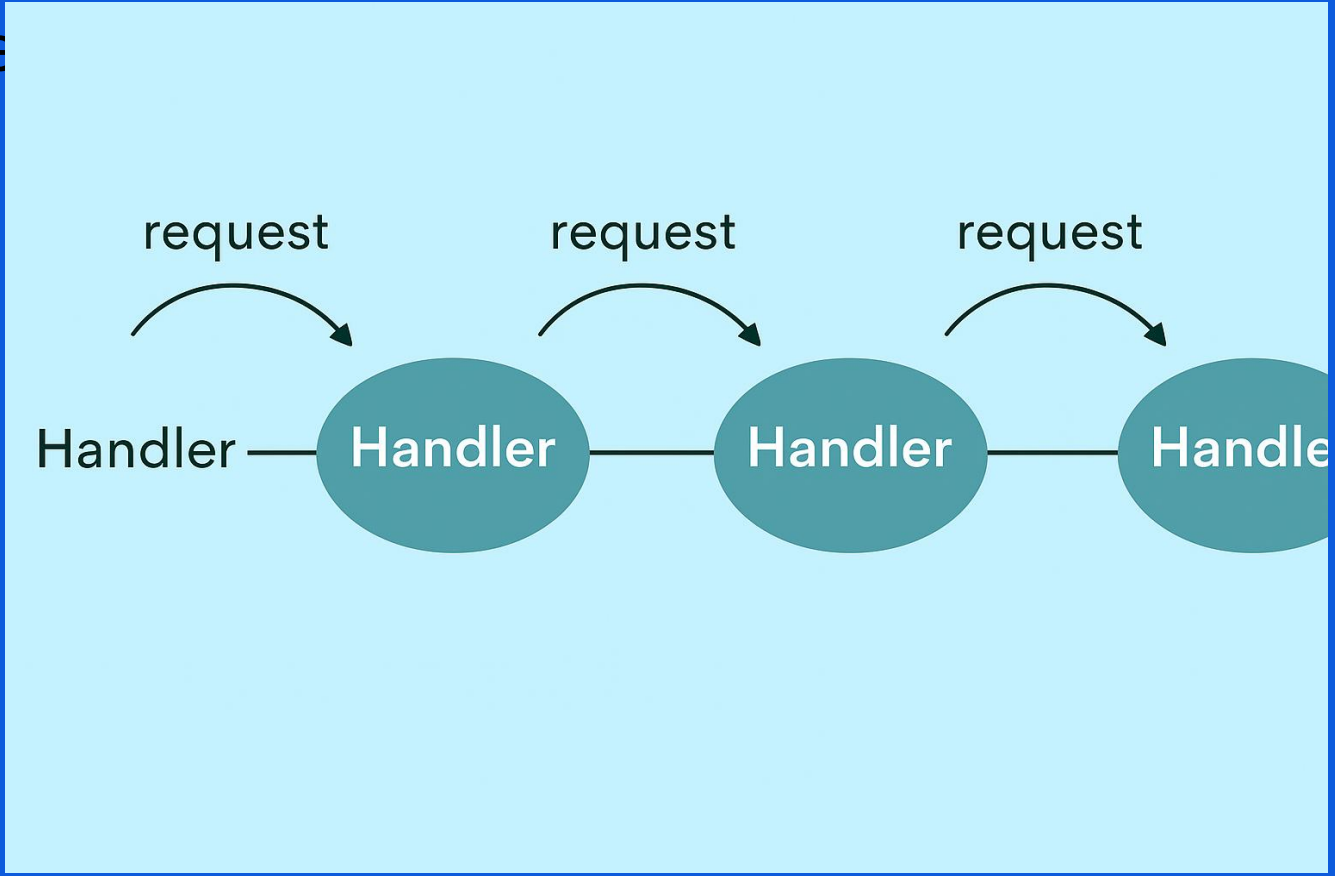
Chain the receiving objects and pass the request along the chain until an object handles it.

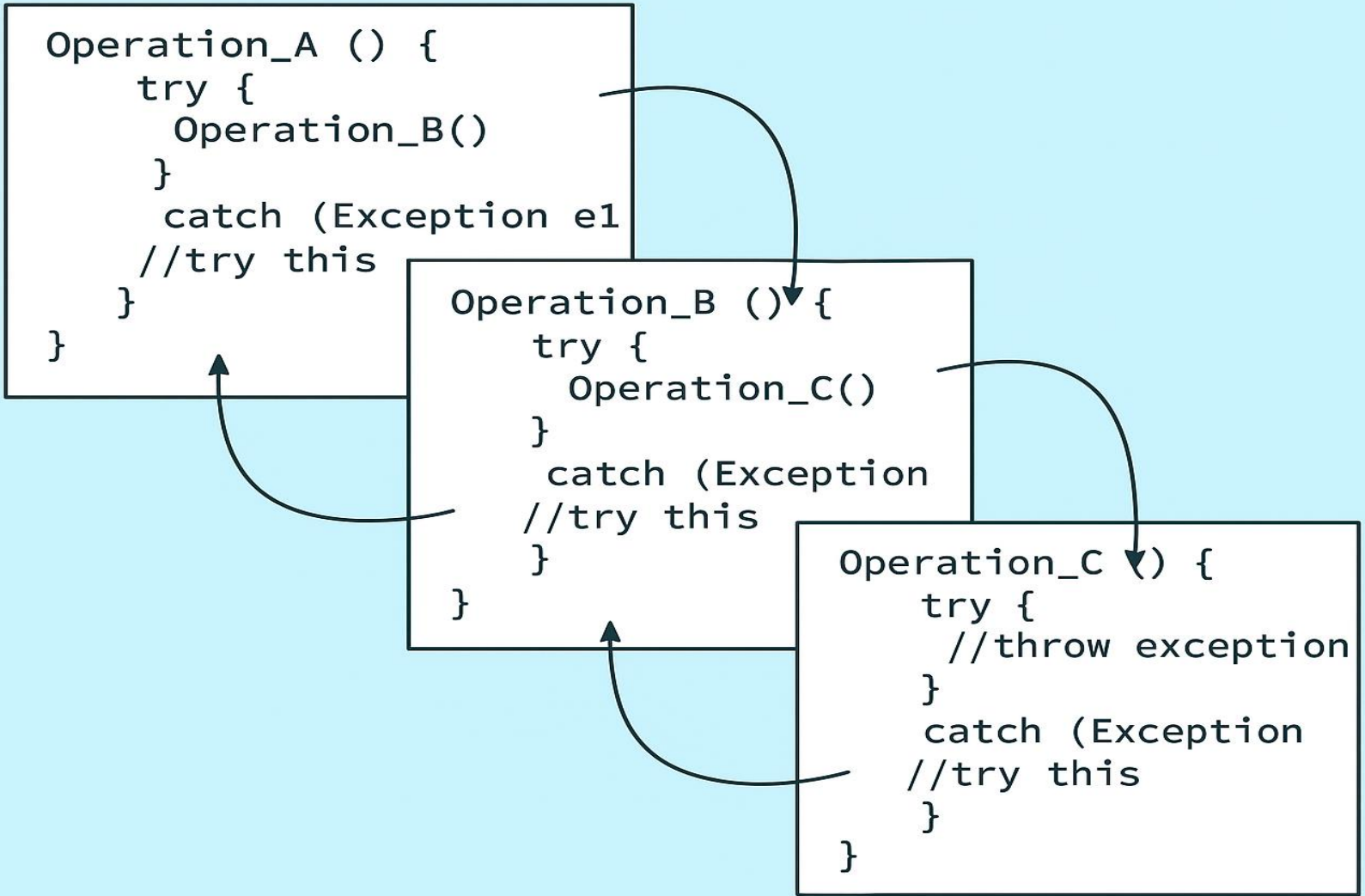
- Ex: Consulting a doctor.

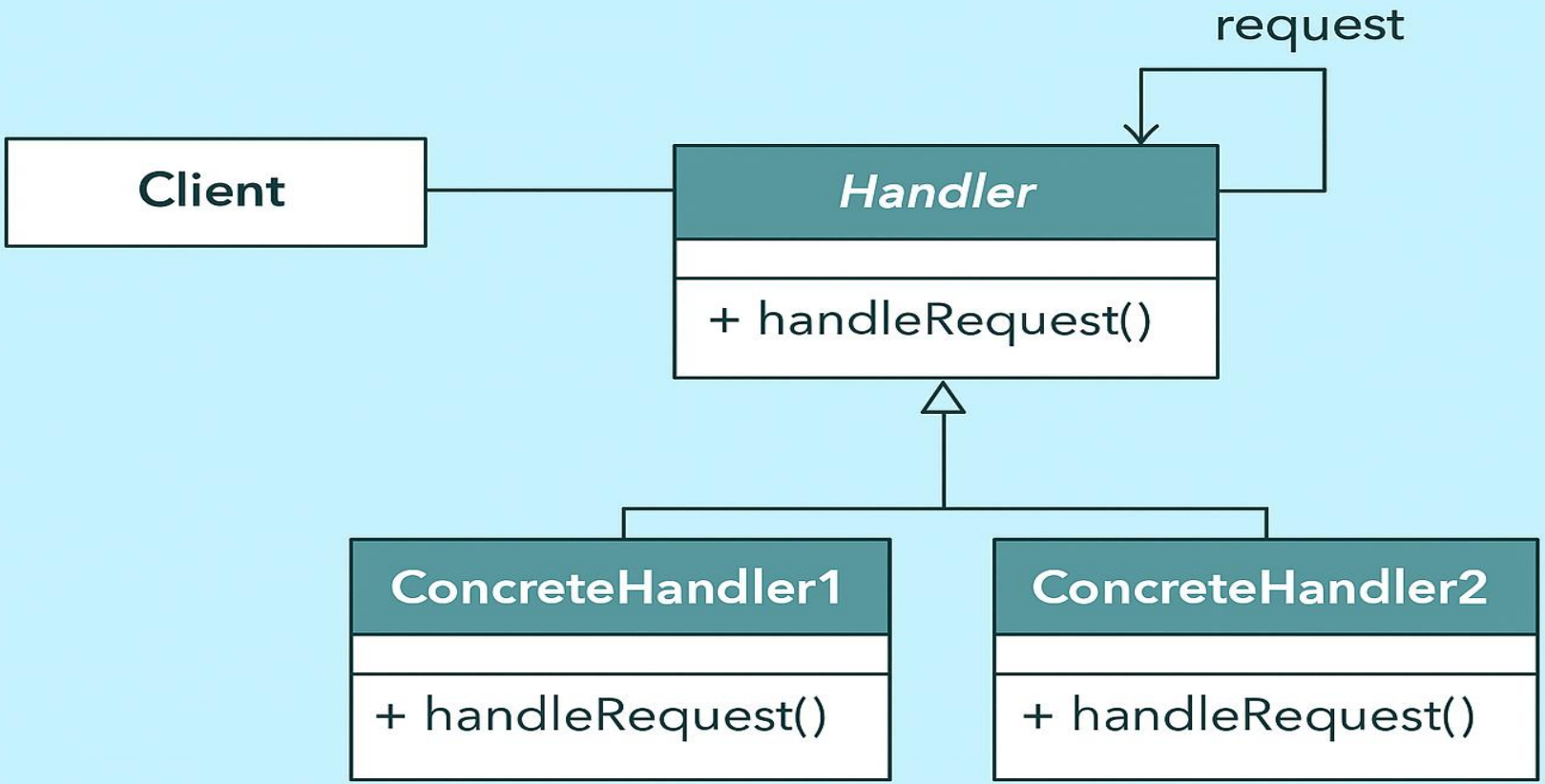
The processing chain may end in either of these two scenarios:

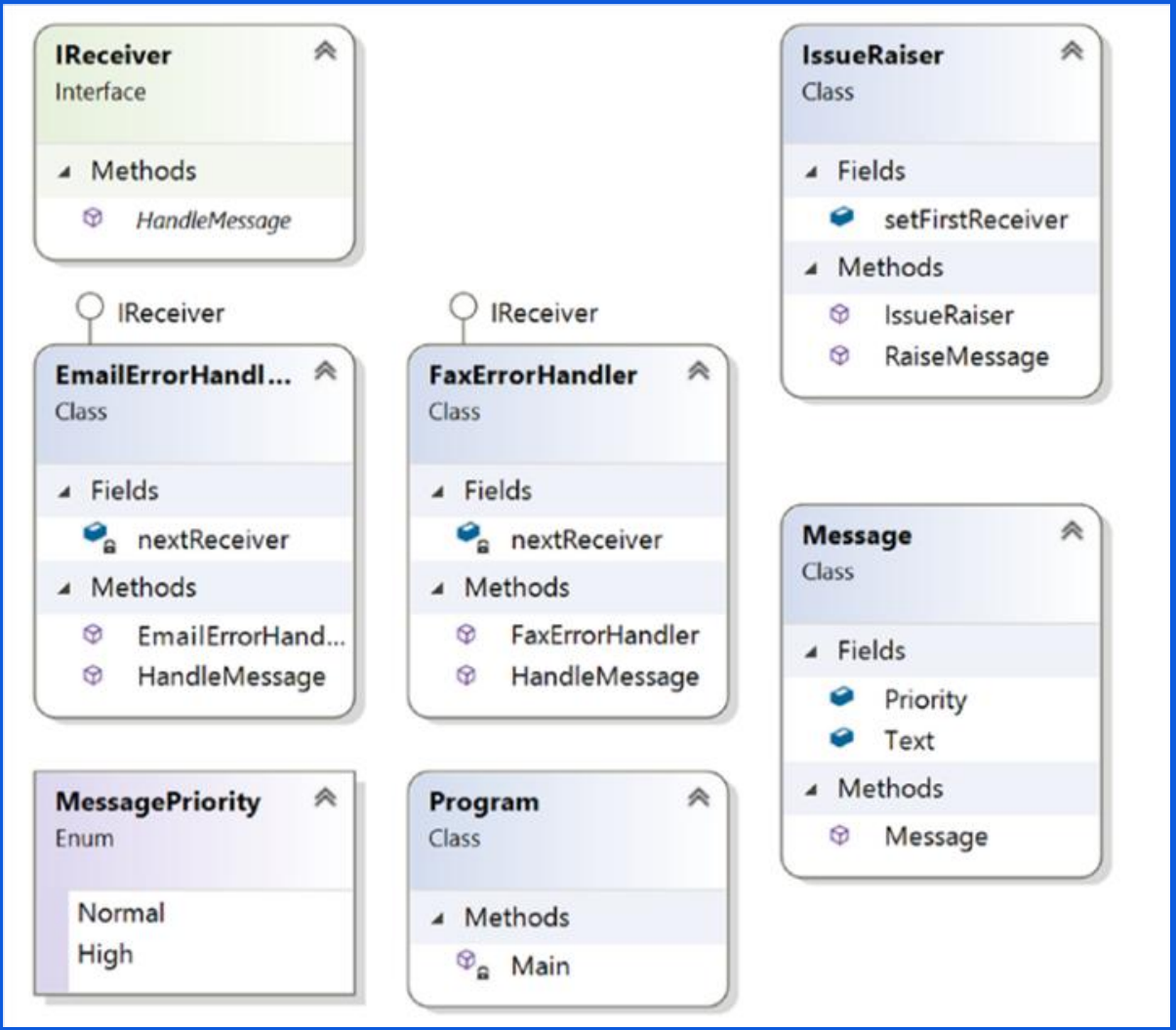
- A handler can process the request completely.
- You have reached the end of the chain.

Chain of Re




Operation_A () {
try {
Operation_B()
}
catch (Exception e1
//try this
}
}
Operation_B () {
try {
Operation_C()
}
catch (Exception
//try this
}
}
Operation_C () {
try {
//throw exception
}
catch (Exception
//try this
}
}


request
Client
Handler
+ handleRequest()
ConcreteHandler1
+ handleRequest()
ConcreteHandler2
+ handleRequest()


IReceiver
Interface
Methods
HandleMessage
IssueRaiser
Class
Fields
setFirstReceiver
Methods
IssueRaiser
RaiseMessage
IReceiver
IReceiver
EmailErrorHandl...
Class
Fields
nextReceiver
Methods
EmailErrorHand...
HandleMessage
FaxErrorHandler
Class
Fields
nextReceiver
Methods
FaxErrorHandler
HandleMessage
Message
Class
Fields
Priority
Text
Methods
Message
MessagePriority
Enum
Normal
High
Program
Class
Methods
Main

Setting up the Priorities



PERFORMANCE



ACCURACY



LESS COMPLEX

Applications

- Email Spam detectors
- Medical Testing
- Troubleshooting
- Classifier Systems
- Event Handling Frameworks
- Security Systems

What are the advantages of using the Chain of Responsibility design pattern?



You have more than one object to handle a request.



The nodes of the chain can be added or removed dynamically. Also, you can shuffle their order.



A handler does not need to know how the next handler in the chain will handle the request. It can focus on its own handling mechanism only.



In this pattern, you are decoupling the senders (of requests) from the receivers.



Can improve High Availability (HA) & Disaster Recovery (DR)

What are the challenges associated with using the Chain of Responsibility design pattern?



There is no guarantee that the request will be handled because you may reach at the end of chain but have not found any explicit receiver to handle the request.



Debugging becomes tricky with this kind of design.



Unnecessary handlers may lead to inefficiency.

Priority can be set.

Bypassing the handlers depending on the information received.

Reflective Questions

- How does the Chain of Responsibility pattern promote flexibility in assigning responsibilities among objects, and what trade-offs might it introduce?
- In what real-world software scenarios could the Chain of Responsibility be more effective than using conditional statements (e.g., if-else chains)?
- How might improper implementation of the Chain of Responsibility lead to performance issues or debugging difficulties, and how can these be mitigated?